

Evaluating LLM Capabilities for Database Design and Programming

Chaitanya Kaushal (A0326730J), Khushali Anil Patel (A0332936Y),
Layana Trevedy (A0332035U), and Prakhar Nagpal (A0318539U)

National University of Singapore, Singapore
<http://www.nus.edu.sg>

Abstract. Large Language Models (LLMs) show strong general performance, yet their reliability on specialist technical curricula remains poorly understood. This paper benchmarks GPT-4o and Llama 3.3 70B on 80 database management systems questions drawn from Ramakrishnan and Gehrke’s standard textbook, evaluating nine prompting strategies and hyperparameter sensitivity under a multi-criteria scoring framework. GPT-4o outperforms Llama across all metrics, achieving a composite score of 70.91 vs. 67.39 and a 148-point Elo advantage. Hallucination rates – 28.52% for GPT-4o and 39.29% for Llama – concentrate on procedural subtopics, pointing to Retrieval-Augmented Generation as the key remediation direction for LLM-assisted DBMS education. These findings offer actionable guidance for DBMS educational systems, automated grading pipelines, and hybrid deployment architectures.

Keywords: Large Language Models · LLM Evaluation · DBMS Benchmarking · Prompt Engineering · Hallucination · GPT-4o · Llama · Elo Rating · Master Composite Score · Retrieval-Augmented Generation

1 Introduction

Relational database systems underpin virtually every modern software application, making proficiency in database design and SQL essential for computing students worldwide. Instructors must evaluate student answers across a wide spectrum – from ER diagram design and SQL formulation to transaction isolation and external sorting algorithms – creating a natural opportunity for LLM-based automation. Yet deploying LLMs in educational settings carries significant risk: a model that fabricates a non-existent PostgreSQL function or misstates second normal form conditions actively propagates incorrect knowledge. Responsible adoption requires quantitative, reproducible evidence of model capability and failure modes before deployment.

1.1 Motivation and Scope

This paper investigates: *to what extent can state-of-the-art LLMs reliably answer the full breadth of questions arising in an introductory DBMS course?* Two production-grade models, GPT-4o (OpenAI) and Llama 3.3 70B (Groq) were

evaluated, representing distinct points on the capability-cost frontier. The question bank is drawn from Ramakrishnan and Gehrke [1], the most widely adopted DBMS textbook in university computing curricula, ensuring ecological validity. The study is framed around four explicit research questions:

- **RQ1.** Which model achieves higher composite performance when evaluated under a multi-criteria rubric covering correctness, quality, and efficiency?
- **RQ2.** How do nine distinct prompting strategies affect each model’s accuracy and consistency and token efficiency?
- **RQ3.** How sensitive is each model’s performance to temperature and top_p hyperparameters, and what configurations are recommended for deployment?
- **RQ4.** What are the dominant failure modes, and what actionable deployment recommendations follow?

1.2 Contributions

This paper contributes: (1) a **fully automated, reproducible evaluation pipeline** with all results publicly available via an interactive dashboard;¹ (2) a **multi-criteria Master Composite Score (MCS)** framework separating DB Correctness, LLM Quality, Prompting Effectiveness, and Efficiency into independently interpretable pillars; (3) an **empirical comparison of nine prompting strategies** yielding model-specific deployment recommendations; and (4) a **systematic hallucination analysis** with topic-level severity stratification motivating RAG-augmented deployment architectures.

2 Background

2.1 DBMS Curriculum and Question Taxonomy

An introductory DBMS course usually covers three areas: *data modelling and query languages* (relational model, ER modelling, relational algebra/calculus, SQL), *physical design* (storage, indexing, external sorting, joins), and *system internals* (query optimisation, transactions, concurrency, recovery).

Questions span a wide range of cognitive demands. Conceptual questions test understanding, while practical ones require SQL queries, ER modelling, normalisation, or multi-step cost analysis. This diversity makes DBMS a challenging, multi-dimensional benchmark that extends beyond SQL-only evaluation.

2.2 Large Language Models and Alignment

Modern LLMs are autoregressive transformer models pre-trained on web-scale corpora via next-token prediction and subsequently aligned to human preferences via RLHF [3]. GPT-4o represents the frontier of proprietary RLHF-aligned models. Llama 3.3 70B [4] is an open-weight model trained with supervised fine-tuning and a lighter RLHF pass, served via Groq’s LPU hardware for low-latency inference.

¹ <https://splendid-custard-947011.netlify.app/>

2.3 Prompting Strategies

Prompting design significantly affects LLM output quality [2,5]. The eight non-trivial strategies evaluated in this work, beyond zero-shot, are: (i) *One-Shot* – a single in-context question-answer pair demonstrating the expected response format; (ii) *Few-Shot* – multiple in-context examples used to further anchor the model’s response style and reasoning pattern [2]; (iii) *Chain-of-Thought (CoT)* – explicit instruction to reason step-by-step before answering [5]; (iv) *Few-Shot-CoT* – combining in-context examples with CoT instructions, shown to substantially improve performance on reasoning-heavy benchmarks; (v) *Self-Consistency* – multiple independent samples with majority vote aggregation [6]; (vi) *Role-Prompting* – expert-persona system prompt instilling domain authority; (vii) *Least-to-Most* – decomposition into incrementally answered sub-questions [7]; (viii) *ReAct* – interleaved reasoning traces and simulated action steps [8].

2.4 Elo Rating Systems in LLM Evaluation

Elo ratings, originally developed for chess, have been adopted for LLM pairwise evaluation in frameworks such as Chatbot Arena [14]. Given current ratings R_A and R_B , the expected score of model A is $E_A = 1/(1+10^{(R_B-R_A)/400})$. After each contest with outcome $S_A \in \{0, 0.5, 1\}$, ratings update as $R'_A = R_A + K(S_A - E_A)$, with $K=32$ ensuring meaningful convergence within 80 contests.

3 Related Work

3.1 General LLM Benchmarking

General-purpose benchmarks such as MMLU [9] include computer science questions but provide only coarse coverage of DBMS topics without textbook alignment or rubric-based scoring. BIG-Bench [10] offers broader task diversity but similarly lacks the conceptual-practical duality of a real DBMS curriculum. This study complements these broad evaluations with fine-grained subtopic analysis across 20 topics tied directly to a canonical textbook.

3.2 SQL and Database-Specific Evaluation

The Spider benchmark [11] evaluates text-to-SQL generation across diverse schemas, focusing on query synthesis from natural language. BIRD [12] extends this with larger and noisier, more realistic databases. While both benchmarks capture SQL generation, they do not assess broader DBMS concepts such as normalisation, indexing, or transaction semantics.

Rajkumar et al. [13] show that GPT-3 performs well on simple queries but struggles with nested subqueries and complex joins. In contrast, the evaluation here is broader: 38.75% of questions focus on practical SQL/schema tasks, while 61.25% assess conceptual DBMS understanding not covered by existing SQL benchmarks.

3.3 LLM-as-Judge Methodology

Zheng et al. [14] introduce MT-Bench and the LLM-as-judge paradigm, showing that GPT-4 aligns closely with human evaluators on open-ended tasks. They also highlight self-preference bias, where a model acting as judge favours its own style; anonymising responses reduces but does not fully remove this effect.

This approach is adopted for two reasons. First, evaluating 1,600 generations requires a scalable and reproducible alternative to human scoring. Second, it avoids subjective bias from human evaluators. To mitigate self-preference bias, answer labels are randomised (A/B), though the limitation of GPT-4o acting as both generator and judge is acknowledged and discussed in Section 5.

3.4 Hallucination in Technical LLM Outputs

Hallucination – the generation of plausible-sounding but factually incorrect content – is a well-documented failure mode of autoregressive LLMs [15]. In the DBMS domain, hallucinated SQL functions or incorrect join cost formulas are not detectable by a non-expert student, motivating a layered regex-plus-judge hallucination detection pipeline.

3.5 LLMs in Computing Education

Prior work has explored LLMs for programming feedback [16] and automated essay grading, but systematic evaluation against textbook question banks with rubric-based multi-criteria scoring in the DBMS domain is not established in the literature. Most existing work is either limited to SQL generation or relies on human evaluation at small scale. This paper fills this gap with a fully automated, reproducible pipeline covering 20 subtopics and three evaluation phases.

4 Methodology

4.1 System Architecture

The evaluation system is built as an asynchronous FastAPI service with seven REST endpoints, supported by a PostgreSQL 16 database using a fully normalised schema.

The system is organised into six main components, each responsible for a specific part of the evaluation pipeline:

1. *LLM Client* : It handles the interaction with model providers (such as OpenAI and Groq), including retries and cost tracking for each API call.
2. *Prompt Builder* : It generates inputs using selected prompting strategies, supports few-shot example retrieval, and creates system-level instructions for role-based prompting.
3. *SQL Harness* : It provides a sandboxed PostgreSQL environment to evaluate SQL outputs where queries are checked for syntactic correction.

4. *Hallucination Analysis* : It uses rule-based checking to identify invalid or fabricated SQL patterns and uses an LLM-based judge to detect more subtle semantic errors.
5. *Composite Scoring Engine* : It combines all the evaluation signals using a predefined rubric into a single score.
6. *Elo Engine* : It uses anonymous model labellings to rank models through pairwise comparisons to ensure unbiased evaluation.

4.2 Question Bank Design

The question bank includes 80 questions drawn from Ramakrishnan and Gehrke [1], covering 20 DBMS subtopics with approximately four questions per topic. Questions are categorised into different sections based on: *difficulty* (Easy: 25 questions, Medium: 40 questions, Hard: 15 questions) and *type* (Conceptual: 49 questions, Practical: 31 questions).

Practical questions are further subdivided by answer format, as each format demands a distinct evaluation rubric. Four practical answer formats are defined as follows:

- **SQL / Query-based**: Questions that require writing SQL queries (e.g., `SELECT`, `CREATE TABLE`). These are evaluated using a sandboxed PostgreSQL environment.
- **ER Diagram Description**: Questions that require structured textual descriptions of entities, relationships, and constraints based on a given scenario.
- **ER Diagram Image**: Questions where the model generates a visual ER diagram, which is compared against the textbook ground-truth figure.
- **Multi-Part**: Questions consisting of multiple sub-parts (e.g., an ER design followed by variations).

All questions include a textbook-aligned reference answer, which serves as the ground truth for both automated evaluation components and the LLM-based judge.

4.3 Master Composite Score (MCS)

The Master Composite Score (MCS) combines all evaluation signals into a single score on a $[0, 100]$ scale:

$$\begin{aligned} \text{MCS} = & 0.50 \times \text{DB_Correctness} + 0.30 \times \text{LLM_Quality} \\ & + 0.15 \times \text{Prompting_Eff} + 0.05 \times \text{Efficiency} \end{aligned} \quad (1)$$

The weights determines the relative importance of each component. DB Correctness (50%) is prioritised as it measures the factual accuracy. LLM Quality (30%) helps to capture how clear, structured, and useful the LLM’s response from a learner’s perspective. Prompting Effectiveness (15%) evaluates how much the performance improves using a particular prompting strategy over a baseline. Efficiency (5%) checks the latency and cost, and is given lesser weight to avoid infrastructure to dominate the overall score.

For single-answer evaluations Prompting Effectiveness and Efficiency are set to a default score of 50 and the final score is mainly determined by DB Correctness and LLM Quality.

DB Correctness Pillar (Weight: 50%): DB Correctness measures how accurate and complete the answer is, with scoring adapted to the question type.

Conceptual Questions: They are evaluated across four aspects: factual accuracy (F), completeness (C), internal consistency and absence of contradiction ($\overline{K}$), and use of domain-specific terminology (S). These are combined as:

$$DB_{conc} = 0.35F + 0.25C + 0.20\overline{K} + 0.20S \quad (2)$$

SQL / Practical Questions: SQL-based questions are evaluated using both automated checks and judge-based scoring. The rubric includes syntax validity (P) via PostgreSQL EXPLAIN, result correctness (F1 score) from sandboxed execution, appropriate usage of clauses like JOIN, WHERE, GROUP BY (A) judged using an LLM-judge, constraint correctness (Con) like appropriate use of primary key, foreign key, NOT NULL, ON DELETE CASCADE analyzed via regex-plus-judge combination, and idiomatic PostgreSQL usage (I) via `pg_catalog` cross-reference. These are combined as:

$$DB_{sql} = 0.15P + 0.30F1 + 0.20A + 0.15Con + 0.20I \quad (3)$$

ER Diagram Description Questions: These are evaluated based on entity coverage (E) - whether all entities are present with correct key attributes, relationship correctness (R) - whether relationships identified with correct cardinalities, constraint handling (Con) i.e. participation constraint (total or partial), and design quality (D):

$$DB_{er} = 0.30E + 0.30R + 0.25Con + 0.15D \quad (4)$$

ER Diagram Image Questions: These use the same rubric, but scoring is based on visual comparison with the reference diagram.

Multi-Part Questions: These are scored per part and combined using a weighted average, with higher weight assigned to the core design component.

LLM Quality Pillar (Weight: 30%) This pillar evaluates how well the model communicates its answer, beyond just correctness:

$$Q = 0.25(1 - HR) + 0.25R + 0.20 \cdot F1 \cdot v + 0.15 \cdot Fmt + 0.15 \cdot Con \quad (5)$$

It captures five aspects: hallucination rate (HR) which penalises incorrect claims, reasoning quality (R), alignment with expected answers (via F1), format compliance (Fmt), and consistency (Con) across variations of the same question. A verbosity factor (v) adjusts scores for overly long or overly brief responses. The final score is scaled to $[0, 100]$.

Prompting Effectiveness Pillar (Weight: 15%). This measures how much a prompting strategy improves performance compared to a zero-shot baseline. It considers accuracy improvement, consistency across runs, token efficiency, reasoning depth, and adherence to expected formats. Scores are normalised to ensure comparability across strategies.

Efficiency Pillar (Weight: 5%). Efficiency captures system-level performance across three factors: time-to-first-token (latency), token generation speed, and API cost. Each component is normalised and combined to produce a final efficiency score.

4.4 Hallucination Analysis

Hallucination detection is implemented as a two-stage pipeline combining rule-based checks with LLM-based evaluation. The *automated layer* detects errors using pattern matching against `pg_catalog`, including fabricated functions, unsupported syntax, and invalid operators. The *judge layer* performs semantic checks by comparing outputs with reference answers and flagging factual inconsistencies. For ER questions, it also identifies domain-specific issues such as incorrect entities, cardinalities, and modelling choices.

Each hallucination is assigned a severity level that determines its penalty:

- **Critical** (3.0): Fundamental errors (e.g., fabricated functions, incorrect cardinalities, invalid ER modelling).
- **High** (2.0): Major conceptual mistakes (e.g., incorrect joins, missing key ER elements).
- **Medium** (1.0): Partial inaccuracies (e.g., imprecise rules, misplaced attributes).
- **Low** (0.5): Minor issues (e.g., overgeneralisation, incomplete explanations).

The overall Hallucination Severity Score (HSS) is computed as: $HSS = \frac{\sum_i w_i}{n \times 3.0}$ where w_i is the severity weight of each detected hallucination and n is the total number of hallucinations.

4.5 Evaluation Phases

The evaluation is conducted in three phases, each designed to measure a different aspect of model performance:

1. **Phase 1 (Baseline Evaluation):** This phase creates the zero-shot baseline by evaluating both models on all 80 questions using default hyperparameters ($T = 0.3$, $\text{top_p} = 0.9$, $\text{max_tokens} = 1024$).
2. **Phase 2 (Prompting Strategy Evaluation):** This phase benchmarks nine prompting strategies on a stratified subset of 20 questions, ensuring balanced coverage across topics and difficulty levels.
3. **Phase 3 (Hyperparameter Sweep):** This phase evaluates the impact of decoding parameters by using temperature $\in \{0.0, 0.7, 1.0\}$ and $\text{top_p} \in \{0.7, 0.85, 1.0\}$ across all 80 questions for both models.

4.6 Elo Rating System

Model performance is also evaluated using an Elo rating system based on pairwise comparisons. In Phase 1, each question forms a head-to-head contest where the judge model selects the better answer between two anonymised outputs.

To eliminate positional bias, responses are labelled as “Model A” and “Model B”, with labels randomly assigned for each comparison. The standard Elo update with $K = 32$ is applied after every contest:

$$E_A = \frac{1}{1 + 10^{(R_B - R_A)/400}}, \quad R'_A = R_A + 32 \times (S_A - E_A) \quad (6)$$

where R is the current rating, E is the expected score, and S is the observed outcome (1 for a win, 0.5 for a tie, 0 for a loss). Both models are initialised with a rating of 1,200. The choice of $K = 32$ ensures that ratings update enough to reflect differences between models within 80 comparisons, but not so much that they become unstable.

4.7 Scoring Consistency and Judge Protocol

To ensure consistent and reproducible evaluation, the judge LLM uses deterministic settings (temperature = 0, `seed` = 42).

Scoring follows a rubric-first approach, where all criteria are reviewed before assigning scores. The judge prioritises factual correctness over fluency, applies consistent strictness across models, and incorporates hallucination penalties before final scoring. Final scores are then checked against predefined bands (Excellent: 9.0–10.0; Good: 7.0–8.9; Acceptable: 5.0–6.9; Partial: 3.0–4.9; Poor: 0.0–2.9)

5 Performance Evaluation

5.1 Overall Leaderboard (Non-ER Questions)

Table 1 presents the final leaderboard aggregating all 1,600 scored runs. Elo ratings and win rates are computed from the 80 Phase 1 pairwise contests only.

Table 1. Final leaderboard sorted by MCS. Elo and Win Rate from 80 Phase 1 pairwise contests.

Model	MCS	DB	Corr.	LLM Qual.	Elo	Win Rate	Halluc.%
GPT-4o	70.91	77.30	74.67	1,274	63.75%	28.52%	
Llama 3.3 70B	67.39	70.43	67.55	1,126	36.25%	39.29%	

GPT-4o leads across all quality metrics, including composite score, correctness, and answer quality, and also achieves a higher Elo rating and win rate. This reflects a consistent advantage in head-to-head comparisons. The Elo gap corresponds to a strong expected win probability, which is broadly aligned with observed results despite sampling variability. In contrast, Llama is significantly more efficient, with lower latency and cost, highlighting a clear trade-off between performance and operational efficiency.

5.2 Phase 1: Zero-Shot Baseline (Non-ER Questions)

Phase 1 establishes the zero-shot baseline under default hyperparameters, providing the fairest comparison of intrinsic model knowledge. Table 2 summarises all primary metrics.

Table 2. Phase 1 zero-shot baseline performance summary (80 questions per model).

Metric	GPT-4o Llama 3.3 70B		Difference
Mean MCS	70.52	67.47	GPT-4o +3.05
Mean DB Correctness	75.29	69.50	GPT-4o +5.79
Mean LLM Quality	75.38	67.85	GPT-4o +7.53
Mean Judge Score (0–10)	7.13	6.42	GPT-4o +0.71
Mean Efficiency Score	55.23	97.23	Llama +42.00
Min MCS	40.23	30.06	GPT-4o higher floor
Max MCS	89.78	91.39	Llama slightly higher ceiling
Avg Latency	9,725 ms	2,831 ms	Llama faster
Avg Cost per Call	\$0.00739	\$0.00079	Llama cheaper
Hallucination Rate	38.75%	58.75%	Llama higher

Across all accuracy metrics, GPT-4o demonstrates a clear advantage, with a higher performance floor and more consistent scores. Llama, however, achieves a slightly higher ceiling, performing competitively on certain pattern-aligned questions. This comes with a trade-off: GPT-4o is slower and less efficient, while Llama offers better latency and cost efficiency.

Topic-Wise Performance Analysis. The evaluation spans 20 DBMS subtopics. Table 3 reports the topic-wise breakdown.

GPT-4o leads in 15/20 subtopics, demonstrating broader and more consistent coverage. Llama outperforms GPT-4o in a few areas, mainly those involving standardised or pattern-driven concepts. Both models struggle on procedurally intensive topics such as evaluating relational operators and concurrency control, which require multi-step reasoning and precise computation. GPT-4o shows its largest gains in areas like external sorting and query evaluation, highlighting stronger handling of multi-step reasoning and explanation.

Difficulty and Question-Type Gradient. Performance differences widen with difficulty. GPT-4o’s win rate rises from 56% on easy questions to 80% on hard questions, indicating stronger scaling on multi-concept tasks. This suggests that the gap is driven more by reasoning ability than by surface-level knowledge. Across question types, GPT-4o performs better on practical tasks than conceptual ones, reflecting a relative strength in structured problem-solving such as SQL composition and schema reasoning.

Table 3. Topic-wise Phase 1 zero-shot MCS (10 of 20 subtopics shown).

Subtopic	GPT-4o MCS	Llama MCS	Δ (GPT-Llama)
Overview of Query Evaluation	82.43	71.00	+11.43
Internet Applications	82.09	74.31	+7.78
Database Application Development	81.40	82.39	-0.99
The Relational Model	76.75	77.66	-0.91
SQL: Queries, Constraints, Triggers	74.39	73.23	+1.16
External Sorting	70.06	55.28	+14.78
Hash-Based Indexing	69.38	61.18	+8.20
Concurrency Control	60.73	55.81	+4.92
Evaluating Relational Operators	56.48	53.94	+2.54
Tree-Structured Indexing	72.48	69.42	+3.06

5.3 Phase 2: Prompting Strategy Comparison (Non-ER Questions)

Phase 2 evaluates nine prompting strategies on a balanced 20-question sample spanning difficulty levels and question types. Table 4 reports the unified comparison across models.

Table 4. Unified prompting strategy comparison across GPT-4o and Llama 3.3 70B. Δ values are relative to each model’s zero-shot baseline (GPT-4o = 70.52, Llama = 67.47).

Strategy	GPT-4o				Llama 3.3 70B			
	Avg MCS	Δ	σ	Rank	Avg MCS	Δ	σ	Rank
Zero-Shot	70.52	—	11.81	#1	67.47	—	12.50	#3
One-Shot	68.83	-1.69	15.43	#4	68.26	+0.79	14.14	#2
Few-Shot	69.79	-0.73	12.66	#3	61.55	-5.92	16.24	#5
CoT	70.08	-0.44	14.31	#2	64.80	-2.67	15.03	#4
Few-Shot-CoT	65.43	-5.09	12.92	#7	69.05	+1.58	14.80	#1
Self-Consistency	65.06	-5.46	14.48	#8	59.54	-7.93	12.12	#8
Role-Prompting	66.15	-4.37	12.89	#6	60.61	-6.86	13.54	#7
Least-to-Most	63.50	-7.02	10.41	#9	60.67	-6.80	13.56	#6
ReAct	66.26	-4.26	11.91	#5	58.63	-8.84	13.16	#9

Three clear patterns emerge. First, GPT-4o is most robust under zero-shot prompting, with minimal variation across strategies (max deviation -7.02). Performance generally declines with added structure especially decomposition-based methods which suggests that external scaffolding can interfere with its internal reasoning.

Second, Llama benefits from structured prompting. Few-Shot-CoT performs best (+1.58 over baseline), followed by One-Shot, while CoT without examples and more complex strategies reduce performance. This indicates that examples, rather than reasoning instructions alone, drive improvements.

Third, Few-Shot-CoT is the only setting where Llama outperforms GPT-4o (+3.62). However, GPT-4o’s zero-shot baseline still exceeds Llama’s best score (by 1.47 MCS), suggesting Llama’s gains are prompt-dependent rather than reflecting stronger underlying capability.

Overall, GPT-4o is more stable and less sensitive to prompting, while Llama relies on carefully designed exemplars to remain competitive. Across both models, overly complex prompting strategies consistently reduce performance.

5.4 Phase 3: Hyperparameter Sensitivity (Non-ER Questions)

Phase 3 sweeps temperature $\in \{0.0, 0.3, 0.7, 1.0\}$ and top_p $\in \{0.7, 0.85, 0.9, 1.0\}$ across all 80 questions.

Table 5. Temperature sweep results. MCS averaged over 80 questions per value. Both models show LOW sensitivity.

Temperature	GPT-4o MCS	GPT-4o σ	Llama MCS	Llama σ
$T = 0.0$	72.48	11.96	67.07	12.85
$T = 0.3$	72.51	12.55	69.14	13.69
$T = 0.7$	69.82	13.12	68.61	13.76
$T = 1.0$	72.07	12.37	68.48	12.95
<i>Sensitivity</i>	LOW		LOW	

Table 6. Top-p sweep results. Both models show LOW sensitivity.

Top-p	GPT-4o MCS	GPT-4o σ	Llama MCS	Llama σ
0.70	73.12	11.85	70.02	13.11
0.85	73.08	12.86	69.16	13.57
0.90	71.22	12.65	67.91	13.55
1.00	73.34	11.71	69.91	13.09
<i>Sensitivity</i>	LOW		LOW	

Both models show low sensitivity to temperature and top-p. Performance varies only slightly across the tested range (within ~ 2 – 3 MCS), indicating that results are stable under different sampling settings. The default temperature ($T=0.3$) performs best for both models, confirming it as a reliable choice. For top-p, the optimal settings diverge: GPT-4o performs best with an unconstrained nucleus (top_p = 1.0), while Llama benefits from a more restricted nucleus (top_p = 0.7).

Overall, the low sensitivity implies that deployment is flexible—reasonable parameter choices lead to consistent performance without significant degradation.

5.5 Hallucination Analysis (Non-ER Questions)

Tables 7 and 8 summarise overall hallucination rates and severity.

Llama shows higher hallucination rates (+10.77 pp) and more errors overall, especially at the critical level (33 vs. 17). In contrast, GPT-4o’s errors are mostly low-severity, indicating minor overclaims rather than major factual mistakes.

Error rates also vary by question type. Both models perform well on definitional topics, while procedural and computation-heavy tasks show the most failures. External Sorting is the most error-prone, followed by Hash-Based Indexing,

Table 7. Overall hallucination statistics, Phase 1 (80 responses per model).

Metric	GPT-4o	Llama 3.3 70B	Difference
Overall Hallucination Rate	28.52%	39.29%	Llama +10.77 pp
Responses with ≥ 1 hallucination	31/80	47/80	Llama +16
Avg hallucinations per response	0.80	1.15	Higher for Llama
Max hallucinations (single response)	6	5	Similar

Table 8. Hallucination severity breakdown (Phase 1).

Severity	Weight	GPT-4o	%	Llama	%	Impact
Critical	3.0	17	27%	33	36%	Highest
Medium	1.0	12	19%	19	21%	Moderate
Low	0.5	35	56%	40	44%	Lowest
Total	—	64	—	92	—	Llama +43.8%

Evaluating Relational Operators, and Concurrency Control—tasks that require multi-step numerical reasoning.

Overall, GPT-4o is more robust and fails conservatively, whereas Llama produces more frequent and severe hallucinations, particularly in multi-step reasoning scenarios.

5.6 ER Diagram Generation and Evaluation

Two ER diagram exercises from Ramakrishnan and Gehrke (Exercises 2.3 and 2.5) were evaluated across Claude Sonnet 4.6 and Gemini 3 Flash Image-Nano Banana 2. Each response was scored on entity coverage, relationship correctness, constraint correctness, and design decisions, aggregated into an MCS. Prompting and evaluation settings were held constant.

Overall Results. Table 9 summarises performance across both exercises. Gemini performs better on Exercise 2.3 (54.05 vs. 37.60), driven primarily by stronger DB correctness despite identical entity coverage. Claude performs slightly better on Exercise 2.5 (42.40 vs. 37.50), where Gemini’s lower LLM quality reduces overall performance. Both models remain weak in absolute terms, with all scores below 55, substantially lower than GPT-4o’s 70.91 on standard DBMS QA.

Table 9. ER diagram evaluation results across both exercises.

Ex.	Model	Score	MCS	DB	LLM	Hall.
2.3	Gemini 3 Flash	4.3	54.05	43	68.5	3
2.3	Claude Sonnet 4.6	1.5	37.60	15	67.0	4
2.5	Claude Sonnet 4.6	2.5	42.40	25	63.0	5
2.5	Gemini 3 Flash	1.8	37.50	15	55.0	5

Dimension-Wise Analysis. Table 10 shows dimension scores for Exercise 2.3. Both models achieve perfect entity coverage (1.0), indicating correct identifica-

tion of schema entities. Claude slightly outperforms Gemini in relationship (0.63 vs. 0.50) and constraint correctness (0.60 vs. 0.40), but this does not translate into higher composite DB scores due to heavier penalties for structural omissions such as missing relationships and incorrect cardinality.

Table 10. ER dimension scores for Exercise 2.3.

Model	Entity	Relation	Constraint	Design
Gemini 3 Flash	1.00	0.50	0.40	0.75
Claude Sonnet 4.6	1.00	0.63	0.60	0.75

Hallucination Analysis. Table 11 shows structural errors across both exercises. Every response contains at least one high-severity error, yielding a 100% severe hallucination rate, substantially higher than Phase 1 DBMS QA (28.52% GPT-4o, 39.29% Llama). Unlike Phase 1, where errors are mostly minor over-claims, ER outputs exhibit structural failures such as missing relationships, reversed cardinalities, and incorrect entity modelling. These errors directly violate schema semantics rather than merely degrading factual precision.

Table 11. High-severity ER hallucinations (both exercises).

Model	Ex.	Type	Description
Claude	2.3	Missing rel.	Work_proj absent
	2.3	Cardinality	Major arrow reversed
	2.5	Entity model	Missing Place/Telephone entities
	2.5	Entity type	Song incorrectly modeled as weak entity
Gemini	2.3	Missing rel.	Work_proj absent
	2.3	Cardinality	major_dept arrow reversed
	2.5	Entity model	Missing auxiliary entities
	2.5	Entity type	Song incorrectly modeled as weak entity

5.7 Results Interpretation

Multi-Metric Consistency. The overall ranking is consistent across MCS, Elo, and Win Rate, all indicating that GPT-4o outperforms Llama on DBMS question answering. MCS measures absolute answer quality, while Elo and Win Rate reflect relative, head-to-head performance.

The gap between GPT-4o’s expected win probability from Elo (71.0%) and its observed win rate (63.75%) is within the expected variance for 80 comparisons and does not indicate any inconsistency between metrics.

Prompting Strategy Interactions with Architecture. The optimal prompting strategy differs across models due to differences in instruction-following abil-

ity. GPT-4o performs best in zero-shot settings, likely because its stronger alignment enables it to retrieve relevant knowledge directly. Additional prompting structure can slightly reduce performance by interfering with these internal priors. In contrast, Llama benefits from explicit guidance. Few-Shot-CoT improves performance by grounding responses with examples, while plain CoT performs worse (-2.67), suggesting that examples also drive improvement.

Self-Preference Bias in Judge Evaluation. Using GPT-4o as both generator and judge introduces a risk of self-preference bias. The model consistently favours responses that align with its own strengths, such as coherence and structured explanations.

While some cases favour Llama these do not fully offset the broader concern of stylistic bias. Randomising answer order reduces positional bias but does not eliminate deeper self-recognition effects. As a result, win rates may not fully reflect objective performance. Future work should use an independent judge model (e.g., Claude or Gemini) to improve evaluation reliability.

Hallucination as Deployment Risk. Hallucination is the primary risk in deploying LLMs for DBMS tutoring. Observed rates (28% for GPT-4o and 39% for Llama) indicate that a substantial portion of responses may contain incorrect or fabricated information.

Errors are most frequent in algorithmic topics such as external sorting, indexing, and concurrency control, suggesting the need for retrieval-augmented generation (RAG) with textbook grounding. The risk is even higher for ER diagram generation, where models often produce structural errors, highlighting the need for stronger validation compared to text-based tasks.

6 Conclusion

This paper presents a systematic three-phase empirical evaluation of GPT-4o and Llama 3.3 70B on 80 DBMS questions from Ramakrishnan and Gehrke, covering 20 subtopics, nine prompting strategies, and a full temperature $\times$ top_p hyperparameter grid. GPT-4o demonstrates consistent superiority: MCS 70.91 vs. 67.39, DB Correctness 77.30 vs. 70.43, a 148-point Elo advantage, and 63.75% head-to-head win rate, with the performance gap amplifying to 80% on hard questions. Llama offers substantial operational efficiency: $3.29\times$ lower latency and $9.1\times$ lower cost. Optimal prompting is model-specific (zero-shot for GPT-4o; Few-Shot-CoT for Llama) and both models exhibit low hyperparameter sensitivity. Hallucination is concentrated on procedural topics, reaching $\approx 81\%$ for Llama on External Sorting, which is a primary deployment risk, motivating RAG augmentation for algorithmic question types.

These findings yield three deployment recommendations: GPT-4o with zero-shot prompting for accuracy-critical batch evaluation; Llama with Few-Shot-CoT for real-time student tutoring; and a hybrid topic-router dispatching procedural questions to GPT-4o and descriptive questions to Llama. Any future

work should expand the model cohort, employ an independent judge to eliminate self-preference bias, and evaluate full RAG-augmented configurations on the highest-hallucination subtopics.

AI Tools Usage Declaration Portions of the writing, including language refinement and LaTeX formatting, were assisted by ChatGPT. Code implementation and debugging support were assisted by Claude. Some aspects of the system design were adapted from CS5242 course materials, with appropriate understanding and modification by the authors. All ideas, analyses, experiments, and conclusions are the authors’ original work. The authors take full responsibility for the contents in this report.

References

1. Ramakrishnan, R., Gehrke, J.: Database Management Systems, 3rd edn. McGraw-Hill, New York (2002)
2. Brown, T.B., et al.: Language models are few-shot learners. In: Advances in Neural Information Processing Systems, vol. 33, pp. 1877–1901 (2020)
3. Ouyang, L., et al.: Training language models to follow instructions with human feedback. In: Advances in Neural Information Processing Systems, vol. 35, pp. 27730–27744 (2022)
4. Touvron, H., et al.: Llama 2: Open foundation and fine-tuned chat models. arXiv preprint arXiv:2307.09288 (2023)
5. Wei, J., et al.: Chain-of-thought prompting elicits reasoning in large language models. In: Advances in Neural Information Processing Systems, vol. 35, pp. 24824–24837 (2022)
6. Wang, X., et al.: Self-consistency improves chain of thought reasoning in language models. In: International Conference on Learning Representations (2023)
7. Zhou, D., et al.: Least-to-most prompting enables complex reasoning in large language models. In: International Conference on Learning Representations (2023)
8. Yao, S., et al.: ReAct: Synergizing reasoning and acting in language models. In: International Conference on Learning Representations (2023)
9. Hendrycks, D., et al.: Measuring massive multitask language understanding. In: International Conference on Learning Representations (2021)
10. Srivastava, A., et al.: Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. arXiv preprint arXiv:2206.04615 (2022)
11. Yu, T., et al.: Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In: Proceedings of EMNLP 2018, pp. 3911–3921 (2018)
12. Li, J., et al.: Can LLM already serve as a database interface? A big bench for large-scale database grounded text-to-SQLs. In: Advances in Neural Information Processing Systems, vol. 36 (2023)
13. Rajkumar, N., Li, R., Bahdanau, D.: Evaluating the text-to-SQL capabilities of large language models. arXiv preprint arXiv:2204.00498 (2022)
14. Zheng, L., et al.: Judging LLM-as-a-judge with MT-bench and chatbot arena. In: Advances in Neural Information Processing Systems, vol. 36 (2023)
15. Maynez, J., et al.: On faithfulness and factuality in abstractive summarization. In: Proceedings of ACL 2020, pp. 1906–1919 (2020)
16. MacNeil, S., et al.: Experiences from using code explanations generated by large language models in a web software development course. In: Proceedings of SIGCSE 2023, vol. 1, pp. 931–937 (2023)